ReservaGol

Explicación del código del programa

Documento que explica, bloque por bloque, cómo funciona el archivo **ReservaGolPrograma.py** (1045 líneas): una aplicación de escritorio hecha con **Python + Tkinter** para gestionar el alquiler de canchas de fútbol, que guarda y consulta todos sus datos en un **servidor PHP/MySQL**.

1. Resumen general

- **Qué hace:** permite iniciar sesión o registrarse, ver los tipos de cancha, reservar un horario, registrar pagos y señas, cancelar reservas y (para el personal) administrar tipos de cancha, horarios, usuarios y facturación.
- **Qué NO hace:** no guarda nada en la computadora. No hay base de datos local ni archivos de datos: cada pantalla pide la información al servidor y cada cambio se envía al servidor.
- **Quién la usa:** tres roles (administrador, empleado y cliente) que ven pestañas y opciones distintas.
- **Librerías:** solo la biblioteca estándar de Python (no requiere instalar nada con pip).

2. Arquitectura: cliente y servidor

El programa es el **cliente**. Toda la lógica de negocio (validar datos, calcular montos, decidir reintegros, controlar permisos) vive en el servidor; el cliente se ocupa de dibujar pantallas y enviar pedidos.

Cada pedido es una llamada a una única dirección (`API_URL`) indicando una **acción** en la URL, por ejemplo `api.php?action=reservas.list`. El servidor responde siempre con JSON y la sesión se identifica con un **token** enviado en el encabezado `X-Token`.

3. Mapa del archivo

Líneas	Bloque	Para qué sirve
1 - 16	Docstring e imports	Descripción del programa y librerías usadas.
18 - 47	Configuración	URL del servidor, colores, fuentes y listas de opciones fijas.
50 - 126	Funciones auxiliares	Formato de dinero y horas, creación de etiquetas, tablas y estado de pago.
128 - 171	ApiError y Api	Comunicación con el servidor y manejo de errores.
174 - 335	Field, CrudPanel, ListPanel	Formularios y tablas genéricos y reutilizables.
338 - 433	SchedulePanel	Pantalla especial para generar y administrar horarios.
436 - 1035	ReservaGolApp	La aplicación: login, registro, panel principal, pestañas, reservas y pagos.

Líneas	Bloque	Para qué sirve
1037 - 1045	main()	Crea la ventana de Tkinter y arranca el programa.

4. Imports y configuración (líneas 1 a 47)

Librerías importadas

Librería	Uso en el programa
json	Convertir datos a JSON para enviarlos y leer las respuestas del servidor.
os	Leer la variable de entorno `RESERVAGOL_API`.
urllib.request / urllib.error	Hacer los pedidos HTTP al servidor y capturar errores de conexión.
dataclasses.dataclass	Definir la clase `Field` de forma compacta.
datetime (date, datetime, timedelta)	Fecha de hoy, validar fechas escritas y avanzar al día siguiente.
pathlib.Path	Ubicar los logos PNG junto al archivo .py.
typing.Any	Indicar que `choices` puede tener varios tipos.
tkinter, messagebox, ttk	Interfaz gráfica: ventanas, cuadros de aviso y widgets con estilo moderno.

Constantes

- **API_URL:** dirección del servidor. Toma el valor de la variable de entorno `RESERVAGOL_API` y, si no existe, usa `http://localhost/reservagol/api.php`.
- **BASE_DIR:** carpeta donde está el script; de ahí se cargan los logos.
- **Colores y fuente** (BG, GREEN, GREEN_DARK, etc.): paleta verde de la aplicación, definida una sola vez para mantener un aspecto uniforme.
- **DIAS, METODOS_PAGO, ESTADOS_CANCHA, CONCEPTOS_PAGO, SI_NO, ROLES:** listas y diccionarios con las opciones fijas que aparecen en los desplegables. Por ejemplo, ROLES traduce el valor interno *admin* a "Administrador".
- **STATS:** define las tres tarjetas de la pantalla de inicio (canchas, reservas confirmadas y clientes), con la clave del dato, el título y el color.

5. Funciones auxiliares (líneas 50 a 126)

Función	Qué hace	Ejemplo
money(value)	Da formato de pesos argentinos con punto de miles.	18000 → `$ 18.000`
hhmm(text)	Recorta una hora a HH:MM.	'17:00:00' → '17:00'
label(...)	Crea un `tk.Label` con colores y fuente ya aplicados.	-
add_field(...)	Apila una etiqueta y un campo de entrada, uno bajo el otro.	-
load_logo(root, file)	Carga un PNG. Si el archivo no existe devuelve None y la app usa texto en su lugar.	-
make_tree(...)	Crea una tabla (`ttk.Treeview`) a partir de una lista (clave, título, ancho).	-

Función	Qué hace	Ejemplo
fill_tree(tree, rows)	Vacía la tabla y la rellena con filas (id, valores).	-
selected_id(tree, msg)	Devuelve el id de la fila elegida; si no hay ninguna, muestra un aviso y devuelve None.	-
cell_value(row, col)	Obtiene el texto de una celda, aplicando una función de formato si la columna la trae.	-
payment_status(row)	Calcula el estado de pago de una reserva.	ver abajo

Lógica de payment_status

Compara lo pagado con el total y con la seña requerida (con una tolerancia de 0,005 para evitar errores de redondeo):

```
if estado != "Confirmada":          -> "—"
si pagado >= total                  -> "Pagada"
si hay seña y pagado >= seña        -> "Seña paga"
si pagado > 0                       -> "Pago parcial"
en otro caso                        -> "Sin pagar"
```

6. Comunicación con el servidor (líneas 128 a 171)

Clase ApiError

Excepción propia. Guarda el mensaje de error y el código de estado HTTP (`status`). Se usa tanto para errores que informa el servidor como para fallos de conexión.

Clase Api

Tiene la URL del servidor y el `token` de sesión (vacío hasta iniciar sesión). Su único método, `call(action, data, timeout)`, hace lo siguiente:

- Arma los encabezados: `Content-Type: application/json` y, si hay sesión, `X-Token`.
- Crea un pedido POST a `URL?action=ACCION` con los datos convertidos a JSON.
- Lo envía con `urlopen`. Si el servidor responde con un código de error (HTTPError), igual se lee el cuerpo de la respuesta, porque ahí viene el mensaje.
- Si no hay conexión (URLError, TimeoutError, OSError) lanza un ApiError con un mensaje claro.
- Interpreta la respuesta como JSON. Si no lo es (por ejemplo, la URL no apunta a api.php) lanza otro ApiError.
- Si la respuesta trae `ok = false`, lanza ApiError con el texto de error del servidor; si no, devuelve el contenido de `data`.

```
Respuesta correcta:  {"ok": true,  "data": ...}
Respuesta con error: {"ok": false, "error": "mensaje"}
```

7. Paneles genéricos reutilizables (líneas 174 a 433)

En lugar de programar cada pantalla de administración a mano, el programa usa clases genéricas que se configuran con datos. Así, "Tipos de cancha" y "Usuarios" son la misma clase con distinta configuración.

Field (dataclass)

Describe un campo de formulario: `key` (nombre del dato en el servidor), `label` (texto visible), `default` (valor inicial), `choices` (None = texto libre; lista = desplegable; función = desplegable cuyas opciones se piden al servidor) y `hidden` (oculta lo escrito, para contraseñas).

CrudPanel: crear, editar y eliminar

Dibuja un formulario con cuadrícula (`per_row` campos por fila), tres botones (Guardar, Limpiar formulario, Eliminar seleccionado) y una tabla con los registros. Se configura con: *entity* (prefijo de las acciones, ej. "tipos"), *id_key* (columna del id), *fields*, *columns*, *noun* (palabra para los mensajes), *name_of* y *warn_of*.

Método	Función
build	Construye formulario, botones y tabla; conecta la selección de fila y la tecla Supr.
load_choices	Rellena los desplegables. Guarda un diccionario {texto visible: valor enviado}.
refresh	Pide `entidad.list` al servidor y llena la tabla.
set_widget / clear_form	Escriben valores en los campos / dejan el formulario con sus valores iniciales y sin registro en edición.
load_selected	Al elegir una fila, pasa sus datos al formulario y recuerda el id en edición (`editing_id`).
read_form	Lee los campos; traduce el texto de los desplegables al valor real.
save	Envía `entidad.save`. Si hay `editing_id`, agrega el id y el servidor actualiza; si no, crea uno nuevo.
delete	Pide confirmación (con advertencia opcional de `warn_of`) y envía `entidad.delete`.

ListPanel: tabla de solo lectura

Muestra los resultados de una acción del servidor y calcula un **total** sumando una columna, que se muestra al pie. Se usa para la Facturación, donde el total cobrado es neto de reintegros.

SchedulePanel: horarios de cada tipo de cancha

- Arriba hay un formulario: tipo de cancha, día (o "Todos los días"), hora desde, hora hasta y duración del turno (60, 90 o 120 minutos).
- **Generar turnos** envía `horarios.generate` y el servidor crea los turnos; informa cuántos horarios nuevos creó.
- La tabla inferior lista los horarios del tipo elegido (`horarios.list`), con su estado Habilitado/Deshabilitado.
- **Habilitar / deshabilitar** (`horarios.toggle`) y **Eliminar** (`horarios.delete`) actúan sobre el horario seleccionado.

8. La aplicación principal: ReservaGolApp (líneas 436 a 1035)

8.1 Inicio y estructura general

- **__init__:** configura la ventana (1080x700, mínimo 900x600), crea el objeto `Api`, carga los logos (login, cabecera e ícono), aplica estilos y muestra el login.
- **is_admin / is_staff:** propiedades que indican si el usuario actual es administrador, o administrador/empleado.

- **setup_styles:** usa el tema "clam" y define el aspecto de pestañas, tablas y botones (incluido el estilo *Primary.TButton* verde).
- **clear:** destruye todos los widgets de la ventana para poder cambiar de pantalla.
- **refreshers:** diccionario que asocia cada pestaña con la función que recarga sus datos.

8.2 El método request: manejo central de errores

Todas las pantallas llaman al servidor a través de `request`, que envuelve `Api.call`. Si algo falla muestra el mensaje en un cuadro de error y devuelve **None**; por eso el resto del código siempre comprueba `if result is None: return`. Caso especial: si el código es **401** (sesión vencida), borra la sesión, avisa y vuelve al login.

8.3 Inicio de sesión y registro

- **auth_card:** arma el fondo verde con logo y una tarjeta blanca; la comparten login y registro.
- **show_login:** campos Usuario y Contraseña (Enter inicia sesión), botón para registrarse, y una línea de estado que indica si el servidor responde (**check_server** hace una llamada "ping" con 4 s de espera).
- **show_register:** formulario de cuenta de cliente (nombre, apellido, teléfono, email opcional, usuario y contraseña repetida). Verifica que ambas contraseñas coincidan antes de enviar `register`; luego vuelve al login con el usuario ya escrito.
- **login:** envía usuario y contraseña; guarda el `token` y los datos del usuario y abre el panel principal.
- **logout:** avisa al servidor (si no hay conexión, igual cierra la sesión local) y vuelve al login.

8.4 Panel principal y pestañas

show_dashboard arma la barra superior (logo, usuario y rol, botón "Cerrar sesión") y un `ttk.Notebook` con las pestañas Inicio, Canchas y Reservas; si el usuario es personal (admin o empleado) agrega Administración. **new_tab** crea cada pestaña con título y subtítulo, y registra su función de recarga. Cada vez que se cambia de pestaña (**on_tab_change** / **refresh_selected**) los datos se vuelven a pedir al servidor, de modo que siempre están actualizados.

8.5 Pestaña Inicio

Muestra tres tarjetas con números (canchas, reservas confirmadas, clientes) tomados de la acción `stats`, un recuadro explicando qué se puede hacer, y, para el personal, un aviso con sus permisos.

8.6 Pestaña Canchas

Tabla con los tipos de cancha: superficie, capacidad, cantidad, precio por hora, seña (%), características (techada o descubierta, con luz o sin luz) y estado. Se puede elegir una fila y pulsar "Reservar cancha seleccionada" o hacer doble clic para abrir la ventana de reserva.

8.7 Nueva reserva (open_booking)

- Abre una ventana modal (`Toplevel` con `grab_set`).
- Si quien reserva es **personal**, primero obtiene la lista de usuarios y arma un desplegable con los clientes activos, porque reserva a nombre de un cliente. Si no hay clientes, avisa cómo crear uno.
- El usuario escribe la fecha (AAAA-MM-DD, por defecto hoy) y elige un horario libre.
- **load_slots** valida la fecha con `strptime` y pide `horarios.list` con el tipo y la fecha. Al abrir la ventana usa *search_ahead*: si no hay horarios ese día, avanza día por día hasta 15 días y salta al primero con lugar, avisándolo.

- **confirm** envía `reservas.create` y muestra el total y la seña requerida.

8.8 Pestaña Reservas

- Tabla con N.º, cliente, cancha, fecha, horario, total, pagado, estado de pago (calculado con `payment_status`) y estado. Los clientes ven solo las suyas; el personal ve todas (esa restricción la aplica el servidor).
- **Cancelar reserva:** solo si está confirmada; pide confirmación y envía `reservas.cancel`. El mensaje avisa que los pagos se reintegran solo si se cancela con la anticipación que fija el complejo; el servidor decide y devuelve un mensaje que se muestra.
- **Registrar pago o seña (open_payment):** abre una ventana con concepto (Seña o Pago), monto y método. El monto se sugiere automáticamente (la seña pedida o lo que falta pagar, sin superar lo pendiente). Envía `facturacion.create` y muestra el número de comprobante y lo que resta pagar. No permite pagar reservas no confirmadas ni ya pagadas por completo.
- **Ver pagos (show_payments):** ventana con todos los movimientos de la reserva (comprobante, fecha, concepto, monto, método y observación).

8.9 Pestaña Administración (solo personal)

Contiene un segundo `Notebook` con cuatro paneles, creados a partir de una lista de configuración:

Sub-pestaña	Clase usada	Detalle
Tipos de cancha	CrudPanel	11 campos (nombre, superficie, medidas, capacidad, cantidad, techada, iluminación, precio, seña %, estado, descripción), con valores iniciales útiles. Precio con formato de dinero.
Horarios	SchedulePanel	Generación y gestión de turnos por tipo de cancha.
Usuarios	CrudPanel	Alta, edición y baja de usuarios. La contraseña vacía significa "sin cambios". Solo el administrador puede elegir el rol; el empleado no ve ese campo. Al eliminar un usuario advierte cuántas reservas y pagos se borrarán con él.
Facturación	ListPanel	Lista de todos los comprobantes y total cobrado neto de reintegros.

8.10 Cierre y arranque

on_close destruye la ventana. **main()** crea la ventana raíz de Tkinter, instancia `ReservaGolApp`, asocia el cierre y ejecuta el bucle de eventos (`mainloop`). El bloque `if __name__ == "__main__"` hace que `main()` corra solo al ejecutar el archivo directamente.

9. Roles y permisos

Función	Cliente	Empleado	Administrador
Ver canchas y reservar	Sí (para sí mismo)	Sí (en nombre de un cliente)	Sí (en nombre de un cliente)
Ver reservas	Solo las propias	Todas	Todas
Registrar pagos y cancelar	Según el servidor	Sí	Sí
Pestaña Administración	No	Sí	Sí
Elegir el rol al crear usuarios	-	No (solo clientes)	Sí

> **Importante:** el cliente solo *oculta* opciones según el rol. La verdadera protección de los datos la hace el servidor, que valida el token y el rol en cada pedido.

10. Acciones del servidor que usa el programa

Acción	Uso
ping	Comprobar que el servidor responde (pantalla de login).
register / login / logout	Alta de cuenta de cliente, inicio y cierre de sesión (el login devuelve token y usuario).
stats	Números de la pantalla de inicio.
tipos.list / .save / .delete	Consultar, guardar y borrar tipos de cancha.
horarios.list / .generate / .toggle / .delete	Listar horarios libres o de un tipo, generar turnos, habilitar/deshabilitar y borrar.
usuarios.list / .save / .delete	Gestión de usuarios.
reservas.list / .create / .cancel	Consultar, crear y cancelar reservas.
facturacion.list / .create	Consultar movimientos y registrar pagos o señas.

11. Flujo típico de uso

- Se abre el programa: aparece el login y, tras 100 ms, se verifica si el servidor responde.
- La persona inicia sesión (o crea una cuenta). El servidor devuelve un token que se guarda en memoria.
- En **Canchas** elige un tipo y pulsa "Reservar". Elige fecha y horario libre y confirma.
- El servidor calcula total y seña. En **Reservas** se registra el pago o la seña; el estado cambia a "Seña paga" o "Pagada" según los montos.
- Si hace falta, se cancela la reserva; el servidor decide si corresponde reintegro.
- El personal, en **Administración**, mantiene canchas, horarios, usuarios y revisa la facturación.

12. Puntos a tener en cuenta

Observaciones sobre el diseño actual del código (no son errores que impidan su uso):

- **Llamadas bloqueantes:** todos los pedidos al servidor se hacen en el hilo de la interfaz. Si el servidor tarda o está caído, la ventana puede quedar "congelada" hasta 4 s (ping) o 10 s (resto de acciones).
- **Sin datos locales:** sin conexión al servidor el programa casi no puede hacer nada, salvo mostrar el login.
- **Reutilización:** gracias a CrudPanel, ListPanel y SchedulePanel, agregar una nueva pantalla de administración requiere sobre todo definir sus campos y columnas, sin escribir una interfaz nueva.
- **Configuración del servidor:** si el servidor no está en `localhost`, hay que cambiar `API_URL` en el código o definir la variable de entorno `RESERVAGOL_API`.
- **Logos:** los archivos `logo_login.png`, `logo_header.png` y `logo_icon.png` son opcionales; si faltan, el programa usa texto.